

SIMATS ENGINEERING

CO2-ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1706

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5

Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I Rekha S K (192511023) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Rekha S K

Annexure A

1.Scenario Based Assignment1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

1. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.

Answer:

- Greedy Best-First Search selects the path with the lowest heuristic value (estimated distance to the goal).
- It ignores the actual travel cost.
- It quickly finds a path but may choose unsafe or blocked routes.
- **Strengths:** Fast, simple, low computation time.
- **Weaknesses:** May produce non-optimal paths and performs poorly when the heuristic is inaccurate or obstacles appear.

2. Explain how A would handle the same situation, including how it balances cost and heuristic.*

Answer:

- A* uses the evaluation function $f(n) = g(n) + h(n)$.
- **$g(n)$:** Actual cost from the start.
- **$h(n)$:** Estimated cost to the goal.
- It considers both travel cost and estimated distance.

- It finds the safest and shortest path when the heuristic is admissible.

3. Analyze the impact of heuristic accuracy on both algorithms.

Answer:

- Accurate heuristics improve search speed and solution quality.
- Greedy Best-First Search depends entirely on the heuristic.
- A* also uses the actual path cost, so it is less affected by heuristic errors.
- Poor heuristics increase search time and may lead to poor solutions.

4. Discuss how dynamic changes (blocked paths) affect search performance.

Answer:

- Blocked paths force the algorithms to recalculate routes.
- Greedy Search may repeatedly choose blocked or unsafe paths.
- A* updates the path using the new costs and obstacles.
- Frequent changes increase computation time.

5. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

Answer:

- *Recommended Algorithm: A Search**
- It balances speed and optimality.
- It considers both actual travel cost and heuristic information.
- It produces safer and more reliable paths in dynamic environments.

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions.

Tasks:

1. Formulate this as an optimization problem and explain why traditional search is inefficient.

Answer:

- Objective: Minimize waiting time, fuel consumption, and congestion.
- Variables: Traffic signal timings.
- Constraints: Traffic flow and road capacity.
- Traditional search is inefficient because the search space is very large and changes continuously.

2. Explain how Hill Climbing would work in this scenario and identify its limitations.

Answer:

- Hill Climbing starts with an initial signal timing.
- It continuously selects a neighboring solution with better performance.
- It stops when no better neighbor exists.
- Limitation: It may get stuck in local maxima and fail to find the best solution.

3. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation.

Answer:

- **Local Maxima:** A good solution that is not the best overall.
- **Plateau:** Many neighboring solutions have the same quality.
- **Ridge:** The best solution requires multiple changes, making improvement difficult.

4. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

Answer:

- Simulated Annealing occasionally accepts worse solutions to escape local maxima.

- Genetic Algorithms use selection, crossover, and mutation to explore many solutions simultaneously.
- Both improve the chances of finding the global optimum.

5. Recommend the best approach and justify your choice based on scalability and adaptability.

Answer:

- **Recommended Approach: Genetic Algorithm**
- Handles very large search spaces efficiently.
- Adapts to changing traffic conditions.
- Produces high-quality solutions with good scalability.

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

1. Explain why this problem requires an online search agent instead of offline search.

Answer:

- The rover does not know the complete environment.
- It learns while exploring.
- Online search allows decisions based on newly discovered information.

2. Analyze the challenges of partial observability and dynamic environments.

Answer:

- The rover cannot see the entire environment.
- New obstacles may appear.
- It must make safe decisions with incomplete information.

3. Describe how the rover builds and updates its internal model.

Answer:

- It collects sensor data.
- It stores explored locations.
- It updates the map whenever new obstacles or paths are discovered.

4. Compare online search with uninformed and informed search strategies.

Answer:

- Uninformed search explores without guidance.
- Informed search uses heuristics with a known map.
- Online search learns and updates information during exploration.

5. Justify the most suitable approach considering uncertainty, adaptability, and safety.

Answer:

- **Recommended Approach: Online Search Agent**
- Works well in unknown environments.
- Adapts to new information.
- Provides safer navigation.

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks:

1. Formulate the problem as a Constraint Satisfaction Problem (CSP).

Answer:

- Each subject is treated as a variable.
- Time slots and exam halls are assigned while satisfying all constraints.

2. Clearly define variables, domains, and constraints with explanation.

Answer:

- **Variables:** Subjects.
- **Domains:** Available time slots and exam halls.
- **Constraints:**
 - No overlapping exams.
 - Limited halls.
 - Faculty availability.
 - Subject order.
 - Student accommodations.

3. Explain how backtracking search works in this scenario.

Answer:

- Assign a time slot to a subject.
- Check constraints.
- If a conflict occurs, return to the previous assignment.
- Continue until all subjects are scheduled.

4. Discuss how constraint propagation (e.g., forward checking) improves efficiency.

Answer:

- Removes invalid choices early.
- Detects conflicts before assigning all variables.
- Reduces unnecessary backtracking.
- Speeds up timetable generation.

5. Analyze real-world challenges and justify the best strategy for scalability.

Answer:

- Large number of students and courses.
- Many scheduling constraints.
- **Best Strategy:** Backtracking with Forward Checking because it reduces the search space and improves efficiency.

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

1. Explain how the Minimax algorithm models this problem.

Answer:

- Minimax considers both the AI's moves and the opponent's moves.
- It selects the move that gives the best possible outcome assuming the opponent plays optimally.

2. Analyze the computational challenges of large game trees.

Answer:

- Large branching factor.
- Huge number of possible game states.
- High memory and computation requirements.
- Slow decision making.

3. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.

Answer:

- Alpha-Beta pruning removes branches that cannot affect the final decision.
- It evaluates fewer nodes.
- It produces the same optimal result as Minimax with less computation.

4. Design an evaluation function and justify the factors considered.

Answer:

Example evaluation function:

- Number of units remaining.
- Health of units.
- Resources collected.
- Territory controlled.
- Distance to objectives.

These factors estimate the strength of the current game state.

5. Recommend a strategy for balancing optimality and real-time performance.

Answer:

- **Recommended Strategy:** Minimax with Alpha-Beta Pruning.
- It finds strong moves while reducing computation.
- It is suitable for real-time games with large search trees.
- It balances optimality and execution speed.